# Laboratorio 2 — Semáforos en XV6

## Clase 6: varios productores y consumidores

Sistemas Operativos — FaMAF, UNC

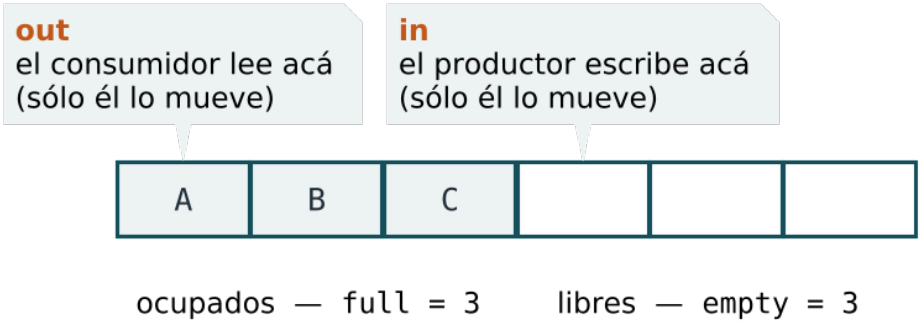
01/10/2026

# Agenda

- Por qué el caso base anda sin mutex
- Qué se rompe con varios
- Un caso real: los pipes
- La entrega final

# Varios productores y consumidores

# Cómo se ve un buffer acotado por dentro




- Dos punteros: `out` dice de dónde leer, `in` dónde escribir.
- Cuando llegan al final, vuelven al principio: por eso es circular, y por eso en el código aparece el `% B`.

# El caso base no necesita mutex

Con **un** productor y **un** consumidor, cada puntero tiene un solo dueño:

| | |
|---|---|
| `in` | la mueve **sólo** el productor |
| `out` | la mueve **sólo** el consumidor |

- No hay ninguna variable que los dos escriban.
- Y nunca trabajan sobre la misma celda: `empty` garantiza que el productor escribe en un lugar libre, `full` que el consumidor lee uno ocupado.

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |

los dos pasan, y está bien: hay lugar para los dos

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` $\rightarrow$ **0** | | 0 |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |
| | lee `in` → **0** ¡el mismo! | 0 |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |
| | lee `in` → **0** ¡el mismo! | 0 |
| | `buffer[0] = B ; in = 1` | 1 |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |
| | lee `in` → **0** ¡el mismo! | 0 |
| | `buffer[0] = B ; in = 1` | 1 |
| ↓ *cambio de contexto* ↓ | | |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |
| | lee `in` → **0** ¡el mismo! | 0 |
| | `buffer[0] = B ; in = 1` | 1 |
| ↓ *cambio de contexto* ↓ | | |
| `buffer[0] = A` pisa a B | | 1 |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |
| | lee `in` → **0** ¡el mismo! | 0 |
| | `buffer[0] = B ; in = 1` | 1 |
| ↓ *cambio de contexto* ↓ | | |
| `buffer[0] = A` pisa a B | | 1 |
| `in = 1` no 2 | | **1** |

# Con dos productores, `in` pasa a ser compartida

| **Productor A** | **Productor B** | `in` |
|---|---|---|
| `down(empty)` ok | | 0 |
| | `down(empty)` ok | 0 |
| los dos pasan, y está bien: hay lugar para los dos | | |
| lee `in` → **0** | | 0 |
| ↓ *cambio de contexto* ↓ | | |
| | lee `in` → **0** ¡el mismo! | 0 |
| | `buffer[0] = B ; in = 1` | 1 |
| ↓ *cambio de contexto* ↓ | | |
| `buffer[0] = A` pisa a B | | 1 |
| `in = 1` no 2 | | **1** |

`full` dice 2 ítems. En el buffer hay 1.

# Lo que falló, y lo que no

- `empty` y `full` hicieron exactamente su trabajo: había lugar para dos, pasaron dos.
- Lo que se corrompió fue `in`, que **ningún semáforo estaba protegiendo**.

Los contadores dan **permiso**, no **exclusión**. Te dicen "'podés entrar"'; si dos reciben permiso a la vez, entran los dos.

Por eso el `mutex` se vuelve obligatorio apenas hay $\mathbf{P} > \mathbf{1}$ o $\mathbf{C} > \mathbf{1}$.

# En nuestro `prodcons` no hay índices...

...pero sí hay una sección crítica: la impresión.

- El buffer es lógico: no hay `in` ni `out` que corromper.
- Pero `printf` hace un `write` **por carácter**, y dos procesos imprimiendo a la vez mezclan las líneas.
- Los tests cuentan **por línea**: una línea partida deja de contar como `produce` o `consume`.

Mismo rol, distinto objeto: en el buffer real el mutex protege la **estructura**; acá protege la **salida**.

# prodcons B N P C

- **P** productores y **C** consumidores: varios `fork`.
- Los **mismos tres semáforos** se comparten entre todos — son globales del kernel, no hace falta nada nuevo.
- Los N ítems se reparten entre los P productores; los N consumos entre los C consumidores.

## Las dos cosas que se complican

**Repartir:** que entre todos produzcan exactamente N, ni uno más.
**Terminar:** que el padre espere a los P+C hijos antes de cerrar.

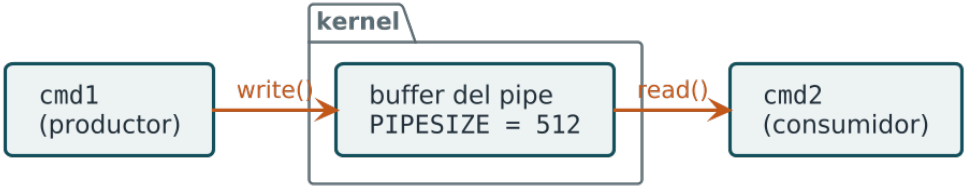
# Un caso real: los pipes

# Esto ya lo usaron

```
$ cat archivo | wc -l
```

- En el Lab 1 implementaron pipes en su shell.
- La salida de un proceso es la entrada del otro: uno produce, el otro consume.
- Entre los dos hay un buffer del kernel de tamaño fijo.

# Un pipe es un buffer acotado




- Si `cmd1` escribe más rápido de lo que `cmd2` lee, el buffer se llena y `cmd1` **se bloquea**.
- Si `cmd2` lee más rápido, el buffer se vacía y `cmd2` **se bloquea**.

# La estructura, en kernel/pipe.c

```
#define PIPESIZE 512

struct pipe {
  struct spinlock lock;
  char data[PIPESIZE];
  uint nread;      // cuántos bytes se leyeron (en total)
  uint nwrite;     // cuántos bytes se escribieron (en total)
  int readopen;
  int writeopen;
};
```

- `nread` y `nwrite` nunca se reinician: crecen para siempre.

# pipewrite: esperar cuando está lleno

```
if(pi->nwrite == pi->nread + PIPESIZE){   // lleno
  wakeup(&pi->nread);
  sleep(&pi->nwrite, &pi->lock);
} else {
  pi->data[pi->nwrite++ % PIPESIZE] = ch;
}
```

- `nwrite - nread` es cuántos ítems hay en vuelo.
- El `% PIPESIZE` convierte el contador infinito en el índice circular.

# piperead: esperar cuando está vacío

```
while(pi->nread == pi->nwrite && pi->writeopen){   // vacío
  sleep(&pi->nread, &pi->lock);
}
...
  ch = pi->data[pi->nread++ % PIPESIZE];
...
wakeup(&pi->nwrite);
```

- `nread == nwrite` significa buffer vacío.
- Y otra vez el `while`, por lo mismo de siempre.

# El mapeo, línea por línea

| **Su `prodcons`** | **`pipe.c`** |
|---|---|
| `B` | `PIPESIZE` (512) |
| `full` (ítems listos) | `nwrite - nread` |
| `empty` (lugares libres) | `PIPESIZE - (nwrite - nread)` |
| buffer lleno → el productor espera | `sleep(&pi->nwrite)` |
| buffer vacío → el consumidor espera | `sleep(&pi->nread)` |
| `up(full)` | `wakeup(&pi->nread)` |
| `up(empty)` | `wakeup(&pi->nwrite)` |
| `mutex` | `pi->lock` |

# Mismo problema, otras primitivas

| **Ustedes** | **xv6** |
|---|---|
| 3 semáforos contadores | 1 spinlock + 2 canales de `sleep` |
| el contador **es** el estado | el estado son `nread`/`nwrite` |

- xv6 no tiene semáforos en el kernel: tiene `sleep`/`wakeup`, que es una condition variable.
- Con una condition variable hace falta llevar el estado aparte; con un semáforo, el contador ya lo es.

El mismo problema se resuelve con cualquiera de las dos.

# ¿Por qué el pipe sí tiene un buffer de verdad?

Porque vive en el kernel.

- La memoria del kernel es **compartida por construcción**: todos los procesos la ven a través de syscalls.
- Dos procesos de usuario, después de `fork`, **no** comparten nada.

Por eso su `prodcons` modela el buffer de forma lógica, y el pipe puede tener 512 bytes reales.

# La entrega final

# make grade-final — 15/15

| Bloque | Tests | Puntos |
|---|---|---|
| `semtest` | los 6 de la intermedia | 6 |
| `pingpong` | args, orden, concurrencia | 5 |
| `prodcons` | args, args inválidos, B=1, B=3 | 4 |
| estrella | `prodcons B N P C` | `@CREDIT` (0) |

- `grade-parcial` tiene que **seguir** en 11/11: no rompan el kernel al agregar `prodcons`.

# Para el 07/10

- ☐ `user/prodcons.c` en `UPROGS`.
- ☐ `make grade-final` → **15/15**.
- ☐ Video de hasta 7 minutos.
- ☐ `make clean` y push.

## Para el video

Si hacen el punto estrella, muestren `prodcons B N P C` con $\mathbf{P} > \mathbf{1}$ y $\mathbf{C} > \mathbf{1}$, y expliquen por qué ahí el `mutex` es imprescindible.

Entrega final: 07/10 23:59